
Evaluating Robustness of Reasoning Models on Parameterized Logical Problems

Naïm Es-sebbani¹ Esteban Marquer² Yakoub Salhi¹ Zied Bouraoui¹

Abstract

Logic provides a controlled testbed for evaluating LLM-based reasoners, yet standard SAT-style benchmarks often conflate surface difficulty (length, wording, clause order) with the structural phenomena that actually determine satisfiability. We introduce a diagnostic benchmark for 2-SAT built from parameterized families of structured 2-CNF formulas, where satisfiability is characterized by the implication graph and can be tuned along interpretable axes. Our generators isolate distinct competencies and failure modes: (i) contradiction-cycle UNSAT cores with controllable size and imbalance, (ii) SAT instances with a prescribed fraction of free variables to control solution multiplicity, (iii) planted backbones that modulate propagation, (iv) late bridge clauses that couple otherwise monotone regions to probe sensitivity to ordering and revision, and (v) symmetry/duplication variants that test abstraction under renaming and redundant structure. We evaluate LLM-based reasoners on decision accuracy and assignment validity, and quantify robustness under semantics-preserving perturbations such as clause reordering, filler clauses, and variable renaming. Across models, we observe sharp performance transitions under targeted structural interventions even when instance size or satisfiability is held fixed, revealing brittleness regimes that are invisible to aggregate SAT accuracy.¹

1. Introduction

Large reasoning models (LRMs) augment standard language models with training- and inference-time mecha-

nisms that encourage explicit multi-step computation, and have achieved strong empirical performance on several domains, including mathematical reasoning and code generation (Patel et al., 2023; Bubeck et al., 2023). Motivated by these gains, recent benchmarks increasingly emphasize combinatorial search and constraint satisfaction, including planning and propositional satisfiability (Wei et al., 2025). While LRMs often outperform conventional LLMs in these settings, recent SAT-based evaluations show that many models remain sensitive to instance hardness, with performance dropping near phase-transition regions; some stronger LRMs are more robust, but still exhibit failures in harder or less familiar formats (Hazra et al., 2025). One interpretation is that some LRMs approximate solver-like procedures through behaviors such as heuristic variable selection, unit propagation, backtracking, self-correction, and candidate selection (Hazra et al., 2025). If such behavior reflects transferable structural reasoning rather than format-specific pattern reuse, then performance should be relatively stable under semantics-preserving transformations and should degrade primarily with controlled changes in structural difficulty.

A central question is therefore how to distinguish transferable algorithmic competence from task-specific pattern reuse. A growing body of evidence points to brittle generalization: models can fail even when instances admit short solutions (Ariyani et al., 2025), exhibit systematic errors on certain relational reasoning classes (Das et al., 2025), and transfer poorly across closely related maze distributions (Valmeekam et al., 2025a). Prior work on Long-CoT distillation further suggests that the global structure of reasoning demonstrations can matter more than the semantic correctness of individual intermediate steps: models remain robust to local content corruptions, but degrade when reasoning steps are shuffled, deleted, or otherwise made structurally incoherent (Li et al., 2025). These findings motivate evaluations that separate structure tracking from reliance on familiar trace formats, local heuristics, and presentation-specific cues.

We study this question through a diagnostic evaluation regime based on parameterized problem families. The goal is not to measure worst-case SAT solving ability, but to avoid the common confound in aggregate SAT-style benchmarks, where instance size, surface presentation, and structural

¹CRIL UMR 8188, Univ Artois, CNRS, France ²GREYC, Université de Caen Basse Normandie, France. Correspondence to: Naïmhe Es-sebbani <essebbani@cril.fr>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

¹The dataset, source code, and prompts are available at <https://github.com/essebbaninaim/reasoning-robustness-llm>.

phenomena can vary together. We instead vary difficulty along explicit structural axes while keeping the verification objective fixed. Concretely, we focus on satisfiability of 2-CNF formulas. Although 2-SAT is polynomial-time solvable, this tractability is a feature of our design: it avoids confounding logical reasoning failures with the absence of an efficient search strategy. At the same time, 2-SAT has a rich and interpretable structural theory. Its implication graph and strongly connected components provide a direct link between graph-theoretic properties and satisfiability. This gives a principled way to manipulate structural phenomena, such as contradiction cores and propagation paths, without changing the underlying verification problem. It also enables controlled instance generation, solver-certified labels and witnesses, and semantics-preserving perturbations.

We design generators that target distinct competencies, including long-range implication tracking, localization of contradiction cores, backbone propagation, and invariance to clause order, redundant clauses, and variable renaming. This fine-grained control enables stress tests beyond aggregate SAT/UNSAT accuracy and reveals brittleness regimes that are not visible under a single instance distribution. Accordingly, our claims are intentionally scoped: the benchmark probes presentation-invariant structural reasoning on parameterized 2-CNF families, rather than general logical reasoning in full.

Our contributions are threefold. First, we introduce a diagnostic 2-SAT benchmark built from parameterized families of structured 2-CNF formulas with controllable implication-graph signatures and semantics-preserving perturbations. Second, we propose an evaluation suite that goes beyond aggregate SAT/UNSAT accuracy by checking both decision correctness and assignment validity, thereby distinguishing correct classification from the ability to construct a valid satisfying witness. We also quantify robustness under targeted interventions such as clause reordering, filler clauses, and variable renaming. Third, we present a systematic empirical study of LLM-based reasoners across these families. Empirically, we find sharp performance transitions and generator-specific brittleness even when instance size is held fixed. We interpret these results as evidence that, on the parameterized 2-CNF families studied here, current LLM-based reasoners do not consistently implement presentation-invariant structural reasoning.

2. Related work

Reasoning in LLMs and LRMs. A large literature evaluates language models as reasoners across inference regimes, including deductive (Clark et al., 2020; Saeed et al., 2021), abductive (Kazemi et al., 2023), commonsense (Tian et al., 2021), and logical or symbolic reasoning (Jiang et al., 2024). Beyond language-centric benchmarks, recent work probes

models on algorithmic and combinatorial tasks that more directly instantiate search and constraint satisfaction, including graph search (Rameshkumar et al., 2025), planning (Valmeekam et al., 2025b), puzzles (Shojaee et al., 2025), propositional satisfiability (Hazra et al., 2025), and graph coloring (Heyman & Zylberberg, 2025). Across these settings, the required inference depth is often a strong predictor of difficulty (Parmar et al., 2024). Methodologically, three common approaches are used to elicit or support reasoning: prompting-based scaffolds such as CoT (Kojima et al., 2022; Saparov & He, 2023) and in-context exemplars (Wang et al., 2023); specialized reasoning models that improve over fine-tuning (Valmeekam et al., 2024); and tool-oriented pipelines that translate problems into symbolic forms executed by external solvers or verifiers (Jiang et al., 2024).

Logical benchmarks and SAT-based evaluation. A prominent line of work studies formal inference when logical statements are rendered in simplified natural language. Clark et al. (2020) report near-perfect accuracy on Horn-rule entailment in such settings, but subsequent analyses find brittleness under distribution shift, consistent with shortcut learning rather than systematic rule-based generalization (Zhang et al., 2023). Related observations hold for satisfiability-style evaluation: Richardson & Sabharwal (2022) consider hard propositional SAT instances presented in simplified language and highlight sensitivity to training data selection and limited extrapolation to larger instances. Another approach seeks robustness by eliciting explicit proof traces, relying on the premise that reliable local steps can compose into longer derivations (Saha et al., 2020; Tafford et al., 2021; Creswell et al., 2023).

Logical reasoning datasets. Benchmarks such as LogiQA (Liu et al., 2020), ReClor (Yu et al., 2020), BoardgameQA (Kazemi et al., 2023), and CLUTRR (Sinha et al., 2019) probe logical and relational reasoning in text, but often entangle formal inference with world knowledge and linguistic priors. In contrast, datasets such as FOLIO (Han et al., 2024a), RuleTaker (Clark et al., 2020), and P-FOLIO (Han et al., 2024b) more directly target knowledge-light logical inference. Logical puzzle corpora provide another controlled avenue (Giadikiaroglou et al., 2024), including ZebraLogic (Lin et al., 2025) and AutoLogi (Zhu et al., 2025). Finally, some recent benchmarks evaluate models on SAT-style inputs more directly, e.g., PARAT (Pan et al., 2025), SATBench (Wei et al., 2025), and SATQuest (Zhao et al., 2026), which frame satisfiability problems as natural language puzzles and leverage classical SAT solvers for verification.

Position. Compared to prior logic and SAT benchmarks, we emphasize *parameterized* instance families with fine-grained structural control and *semantics-preserving perturbations* (e.g., clause order, filler clauses, and renaming

invariances). This differs from SAT-in-natural-language settings that primarily vary surface realizations of fixed instances, and from proof/trace-based evaluations that require models to emit (and be graded on) explicit derivations. Instead, we keep the verification objective unchanged and probe whether models' decisions and assignments remain stable under transformations that preserve satisfiability while targeting specific structural competencies. Focusing on 2-SAT enables mechanistic instance design because satisfiability is characterized by the implication graph, allowing attribution of failures to concrete graph phenomena rather than to instance size or surface form alone.

3. Problem setting

We study the satisfiability of 2-CNF formulas and design parameterized instance families whose difficulty can be tuned through interpretable structural properties. Throughout, let $\mathcal{V} = \{x_1, \dots, x_n\}$ be a finite set of Boolean variables, and let $\mathcal{L} = \mathcal{V} \cup \{\neg x \mid x \in \mathcal{V}\}$ be its associated set of literals. Given a literal ℓ , we write $\bar{\ell}$ for its complementary literal. We use the usual definitions $\ell \rightarrow \ell' := \bar{\ell} \vee \ell'$ and $(\ell \leftrightarrow \ell') := (\ell \rightarrow \ell') \wedge (\ell' \rightarrow \ell) \equiv (\bar{\ell} \vee \ell') \wedge (\bar{\ell}' \vee \ell)$.

A 2-CNF is a conjunction of clauses, each clause a disjunction of at most two literals. A literal in a 2-CNF formula is *pure* if it appears with only one polarity. $n := |\mathcal{V}|$ denotes the number of variables and $m := |\mathcal{C}|$ the number of clauses.

Implication graph for 2-SAT. To decide satisfiability of a 2-CNF formula, we associate with it an implication graph on vertex set $\mathcal{L}$. Each clause $(\ell_1 \vee \ell_2)$ contributes the edges $\bar{\ell}_1 \rightarrow \ell_2$ and $\bar{\ell}_2 \rightarrow \ell_1$. We use $l \rightsquigarrow l'$ to denote that there is a path from l to l' in the implication graph.

A 2-CNF formula is unsatisfiable (UNSAT) iff there exists a variable x such that x and $\neg x$ belong to the same Strongly Connected Component (SCC) of the implication graph (equivalently, $x \rightsquigarrow \neg x$ and $\neg x \rightsquigarrow x$).

Why 2-SAT? Our choice of 2-SAT is motivated by controllability rather than worst-case hardness. The implication-graph characterization gives a direct and interpretable mapping from graph structure to satisfiability, allowing us to vary structural properties while keeping formula size and the verification objective controlled. This is less direct in other tractable fragments such as Horn-SAT, where satisfiability is determined by forward chaining and does not provide the same SCC-based decomposition used by our generators. Moving to harder classes such as 3-SAT would introduce a different confound: failures could reflect either weak structural reasoning or the absence of an efficient search strategy. By contrast, 2-SAT lets us focus on structural tracking and invariance under controlled perturbations.

Difficulty controls. For each generator, we vary the instance

structure and its presentation (clause order, variable renaming), which does not change satisfiability but can stress heuristic sensitivity. For UNSAT generators, we mainly vary (i) the structure of the implication-graph witness of unsatisfiability (the *UNSAT core*) and (ii) the number of *filler* clauses, i.e., clauses that can be satisfied independently of the core and thus do not contribute to the contradiction. For the two SAT generators that parameterise *critical variables*, we vary (i) the number of critical variables, namely backbone variables in the Backbone generator and free variables in the EquivalenceCore generator, (ii) the total number of variables, and (iii) the number of pure literals (literals that occur with only one polarity and hence do not constrain satisfiability). The remaining two generators (ImplicationCycle and MonoBridge) do not parameterise critical variables and instead control difficulty via structural parameters of the implication graph, namely the cycle split position and the bridge clause placement, respectively.

3.1. UNSAT Generator using Implication Cycles

We pick distinct variables $v_1, \dots, v_L$ and for each we pick a literal $\ell_i \in \{v_i, \neg v_i\}$ for $i = 1, \dots, L$. Then, we construct a directed cycle in the implication graph:

Forward part: $\ell_1 \rightarrow \ell_2 \rightarrow \dots \rightarrow \ell_k \rightarrow \bar{\ell}_1$,

Backward part: $\bar{\ell}_1 \rightarrow \ell_{k+1} \rightarrow \dots \rightarrow \ell_L \rightarrow \ell_1$.

Encoding each implication $\ell \rightarrow \ell'$ as the single 2-clause $\bar{\ell} \vee \ell'$ yields a core that forces both $\ell_1 \rightarrow \bar{\ell}_1$ and $\bar{\ell}_1 \rightarrow \ell_1$, hence UNSAT. The construction above contains one clause per displayed implication. The forward part contributes k clauses; the backward part contributes $(L - k + 1)$ clauses, for a total of $L+1$ clauses.

Parameters controlling difficulty. Difficulty is controlled by four parameters: (i) the *cycle split* (L, k) , where L is the number of literals in the core cycle and k is the position of the twist to $\bar{\ell}_1$ (equivalently, the *imbalance* k/L , with extremes $k = 1$ and $k = L$); (ii) the number of *filler clauses* r that are satisfiable independently of the core (controlling size/density); (iii) the *presentation*, i.e., whether core clauses are ordered along the implication chain or shuffled and interleaved with fillers; and (iv) the *total number of variables* $n \geq L$, with variables outside the core appearing only in fillers (controlling the ratio L/n). The key structural difficulty is the imbalance k/L : when it is close to 0 or 1 (e.g., $k = 1$ or $k = L$), one direction of the contradiction cycle is long while the other is short, forcing long-range implication tracking before closing the cycle.

3.2. SAT Generator via Free Variables

We choose a set $F \subseteq \mathcal{V}$ of *free variables* with $|F| = f$, meaning that the values of variables in F can be chosen arbitrarily. For every remaining variable $y \in \mathcal{V} \setminus F$, we choose

a parent $p(y) \in F$ and a literal $\tau(y) \in \{p(y), \neg p(y)\}$, and enforce $y \leftrightarrow \tau(y)$. Each such definition contributes two binary clauses (two implications). Every assignment to F extends uniquely to a satisfying assignment on all of $\mathcal{V}$.

Parameters controlling difficulty. The number of variables n and free variables f determines the model count 2^f . The *free-to-bound ratio* $f/(n - f)$ controls coupling strength: smaller ratios induce stronger propagation from a small free set. We also vary the *presentation order*, placing definition clauses early or late in the clause list to probe order sensitivity. The formula has $2(n - f)$ clauses. This generator isolates sensitivity to solution multiplicity, through f , and to the amount of propagation required to produce a valid assignment.

3.3. Backbone-based Method

The goal is to construct a satisfiable 2-CNF that is the conjunction of: (i) clauses that infer a planted *backbone* assignment, i.e., an assignment on a subset of variables that is forced in every satisfying assignment (these variables take the same truth value in all solutions), and (ii) a remainder consisting only of monotone 2-clauses (all literals have the same polarity). We choose a backbone $B \subseteq \mathcal{V}$ with a planted assignment $b : B \rightarrow \{0, 1\}$, and let $F = \mathcal{V} \setminus B$ be the free part. We introduce a fresh auxiliary variable a_x for each $x \in B$. The formula is a conjunction of two parts:

$$\Phi^* := \underbrace{\Phi_{\text{backbone}}}_{\text{infers } b \text{ using only 2-clauses}} \wedge \underbrace{\Phi_{\text{mono}}}_{\text{monotone 2-clauses over } F}.$$

(A) Backbone inference. For every $x \in B$, we add the following clauses using a fresh auxiliary a_x :

$$\begin{aligned} \text{Case } b(x) = 1: & \quad (x \vee a_x) \wedge (x \vee \neg a_x), \\ \text{Case } b(x) = 0: & \quad (\neg x \vee a_x) \wedge (\neg x \vee \neg a_x). \end{aligned}$$

Each pair is logically equivalent to the unit clause it emulates (respectively x or $\neg x$), but uses only binary clauses; thus it fixes x to $b(x)$ in all satisfying assignments.

(B) Monotone remainder. We choose an orientation $\pi \in \{+, -\}$ and add only 2-clauses over variables in F :

$$\begin{aligned} \text{Case } \pi = - : & \text{ add clauses } (\neg y \vee \neg z) \text{ for } y, z \in F, \\ \text{Case } \pi = + : & \text{ add clauses } (y \vee z) \text{ for } y, z \in F. \end{aligned}$$

No variable from B appears in Φ_{mono} . If $\pi = -$, setting all $y \in F$ to 0 satisfies Φ_{mono} ; if $\pi = +$, setting all $y \in F$ to 1 does. Together with the pinned backbone, Φ^* is satisfiable.

Parameters controlling difficulty. Let $|B| = \beta n$ and m_{mono} be the number of monotone clauses added over F . The part Φ_{backbone} has exactly $2|B|$ clauses and $|B|$ auxiliaries. The total clause count is $|\mathcal{C}| = 2|B| + m_{\text{mono}}$. Difficulty is tuned via the *backbone fraction* β , *monotone density* relative to possible pairs $\frac{2m_{\text{mono}}}{|F|^2}$, and *orientation* π .

3.4. Monotone Split with Moving Cross-Polarity Bridge

We construct a 2-CNF that consists of (i) a positive-monotone part over variables P (only clauses of the form $(p \vee p')$ with $p, p' \in P$), (ii) a negative-monotone part over variables N (only clauses of the form $(\neg q \vee \neg q')$ with $q, q' \in N$), with $P \cap N = \emptyset$, and (iii) a single bridge clause that couples both parts $(\neg p^* \vee q^*)$ with $p^* \in P$ and $q^* \in N$. The bridge is inserted at a chosen position in the clause list so we can probe when a solving method backtracks as it scans clauses.

Parameters controlling difficulty. The number of clauses is $|\mathcal{C}| = m_+ + m_- + 1$ with m_+ and m_- the number of positive and negative clauses, plus the bridge. Densities $m_+/|P|$ and $m_-/|N|$ and the proportion of each part control how strongly each monotone part biases decisions. *Bridge position* s probes sensitivity to late coupling and potential revision during clause scanning; satisfiability is invariant under s . In particular, we parametrize generation with the *overall variable density* $\frac{|P|+|N|}{2|\mathcal{C}|}$, the relative size of the positive part $m_+/|\mathcal{C}|$, and the position $s \in [1, |\mathcal{C}| - 1]$ of the bridge among the clauses.

3.5. Symmetry/Redundancy Probe

Given a base 2-CNF Ψ over variables $\mathcal{V}$, we build a new instance $\Phi := \Psi \wedge \rho(\Psi)$, where ρ is a bijective renaming from $\mathcal{V}$ to a fresh, disjoint set $\mathcal{V}'$ (so $\mathcal{V} \cap \mathcal{V}' = \emptyset$). Thus Φ is the concatenation of two isomorphic, independent copies of Ψ . Comparing a model on Ψ versus on Φ probes whether the model detects the repeated structure and how its behavior is affected by it. We do not assume that a model should exploit the symmetry to reduce computation, since doing so requires identifying the correspondence between the two copies.

Properties (independence and symmetry). *Independence.* $\mathcal{V}$ and $\mathcal{V}'$ are disjoint, so there is no logical interaction. *Automorphism.* Swapping $\mathcal{V} \leftrightarrow \mathcal{V}'$ (i.e. applying ρ or ρ^{-1}) leaves Φ invariant. Φ has a nontrivial symmetry. *Satisfiability.* Φ is SAT iff Ψ is SAT. If Ψ has M models, then Φ has M^2 models (Cartesian product across copies). If Ψ is UNSAT, so is Φ . We therefore use this construction to measure sensitivity to repeated structure, e.g., whether duplication degrades accuracy and whether reasoning traces show any cross-referencing between the two copies, rather than as a test of computational efficiency.

Variants and extensions. *More copies.* We use $d > 2$ disjoint renamings to form $\Psi \wedge \rho_2(\Psi) \wedge \dots \wedge \rho_d(\Psi)$, probing scaling under repeated symmetry. *Copy-wise shuffles.* Randomly permute the clause order within each copy; or apply a variable permutation per copy to avoid accidental alignment in indices. *Trigger placement.* If Ψ has a late trigger clause that forces a contradiction, we place its renamed counterpart

early (or vice versa) to study sensitivity across copies.

Scope of renaming The renaming probed in this section is *cross-copy* renaming: the second copy uses a variable set disjoint from the first, so models see the same logical content under entirely fresh symbols. A second, complementary form of renaming is exercised across the six template verbalizers (Section G): the same underlying variable is mapped to different entity types, including letters, person names, room numbers, and team labels, yielding entity-level renaming at a fixed clause structure. We do *not* test *within-instance* variable renaming, such as permuting variable indices while holding template, clause order, and other surface attributes fixed. Such a probe would isolate sensitivity to the surface identity of individual variables rather than to their structural role; we leave this finer-grained test to future work.

4. Dataset Construction

Verbalizers. We convert each 2-CNF into text using two approaches as illustrated in Fig 10. The *template verbalizer* deterministically renders each clause into a short natural-language statement, using a fixed mapping from variables to entity names within a theme. We use 6 different templates to reformulate the SAT problem into a more practical problem: a direct verbalization of how the clause reads (logic), letter passing between people (letter), assignment of people to a blue and a red team (team), affinities between invitees to constrain a social gathering (social), room lighting rules (room), door locking (door). Each template maps a clause to a fixed sentence pattern with named entities (e.g., $\neg a \vee b$ becomes “either a is on the Blue team or b is on the Red team”). The mapping is deterministic and invertible, so the logical content of each clause is preserved by construction.

The **LLM verbalizer** uses an LLM to rewrite each clause as a short narrative paragraph. We present the LLM with the two literals as natural-language statements derived from a thematic entity mapping (e.g., “the butler is innocent”, “the detective is the murderer”) and ask it to produce a paragraph expressing their disjunction. We use three themes (spy, heist, detective) to guide stylistic context. To prevent information loss, each paragraph is checked by a two-step LLM validator: it (i) extracts the two referenced entities and (ii) infers whether each is stated positively or negatively. We compare the recovered clause to the original; mismatches trigger regeneration (up to three attempts), after which we fall back to a deterministic template. On the augmented dataset, 95.4% of clauses validate on the first attempt and only 3.3% use the fallback, indicating that the verbalizer is typically semantically faithful.

Choice of generation parameters. To generate the dataset, we use the parameters described in Section 3 for each generator. To facilitate sampling, each parameter is designed

to be in $[0, 1]$. In our experiments, for each parameter, we split its possible values into thirds and generate data using an easy to manipulate value close to the center of each interval of the parameter space ($0.2 \in [0; \frac{1}{3}]$, $0.5 \in [\frac{1}{3}, \frac{2}{3}]$, $0.8 \in [\frac{2}{3}, 1]$). When studying the impact of each parameter, we considered 4 additional values that correspond to the bounds of those intervals ($0, \frac{1}{3}, \frac{2}{3}, 1$). For generators where more than one major difficulty parameter is identified, we vary only one parameter at a time and set the others to 0.5 to minimize the number of settings to evaluate. When generating data, the number of generated clauses is the main generation parameter, but minor differences might appear depending on the specifics of each generator. For instance, some generators are unable to generate an odd number of core clauses. Hereafter, the number of clauses is given with a ± 1 tolerance to best match the target difficulty.

Pipeline. For a given generator parameter combination, we generate 10 formulas and produce, for each of them, either 6 verbalizations for the template verbalizer or 3 for the LLM verbalizer (only in the ablation study Section 5.5).

5. Experiments

Table 1 reports results for seven public reasoning models (14B to 120B) across the four main generators, with all generator-specific difficulty parameters fixed to 0.5 and only the number of clauses varied. Model details appear in Appendix Table 3. For each setting, we sample 10 formulas and generate 6 template verbalizations per formula (60 prompts). We parse the SAT/UNSAT decision and, when the model predicts SAT, an assignment, counting truncated or unparseable outputs as incorrect. Ablations focus on four models (Phi4-reasoning, Phi4-reasoning-plus, Qwen3-Next-80B, GPT-OSS-120B) under the same protocol unless noted. Beyond this open-weight evaluation, we also report (i) per-generator, $|C|$ results for three proprietary reasoning models (o3-mini, o4-mini, GPT-5.4-mini) in Section N, (ii) an ablation of GPT-OSS-120B across the low/medium/high reasoning-effort in Section O, and (iii) a tool-calling variant of GPT-5.4-mini with PySAT code execution in Section P.

Number of clauses as the main difficulty parameter. Increasing $|C|$ simultaneously increases (i) input length and parsing burden, and (ii) the amount of structure the model must integrate to reach a consistent global conclusion. We therefore treat $|C|$ as the primary scaling axis. Importantly, $|C|$ conflates core clauses (that witness SAT/UNSAT) with non-core clauses; in section 5.2 we decouple these to test whether failures are driven by reasoning depth in the core or by distractor load. Phi4-reasoning-plus frequently produces outputs that exceed our maximum output budget, leading to truncated or unparseable answers even on moderate sizes. We report these cases as incorrect (per the protocol above). Truncation rates are in Appendix Table 4.

Evaluating Robustness of Reasoning Models on Parameterized Logical Problems

Generator	Model C	Llama-3.3 70B-Instruct		OLMo-3 32B		Phi4 reasoning		Phi4 reasoning-plus		QwQ 32B		Qwen3-Next 80B		GPT-OSS 120B	
		Sat.	Wit.	Sat.	Wit.	Sat.	Wit.	Sat.	Wit.	Sat.	Wit.	Sat.	Wit.	Sat.	Wit.
ImplicationCycle	5	0.0	—	98.3	—	83.3	—	95.0	—	98.3	—	100.0	—	91.7	—
	10	1.7	—	95.0	—	83.3	—	75.0	—	96.7	—	98.3	—	95.0	—
	15	15.0	—	96.7	—	61.7	—	40.0	—	68.3	—	96.7	—	93.3	—
	20	18.3	—	88.3	—	41.7	—	15.0	—	65.0	—	96.7	—	93.3	—
	50	11.7	—	65.0	—	3.3	—	0.0	—	40.0	—	81.7	—	56.7	—
	75	13.3	—	50.0	—	1.7	—	0.0	—	48.3	—	73.3	—	45.0	—
	100	20.0	—	51.7	—	0.0	—	0.0	—	45.0	—	76.7	—	40.0	—
EquivalenceCore	10	100.0	1.7	100.0	93.3	100.0	80.0	100.0	86.7	100.0	86.7	100.0	91.7	100.0	83.3
	15	98.3	0.0	100.0	55.0	90.0	35.0	95.0	35.0	96.7	45.0	100.0	53.3	100.0	46.7
	20	96.7	0.0	91.7	56.7	86.7	21.7	83.3	38.3	86.7	43.3	100.0	63.3	100.0	50.0
	50	93.3	0.0	83.3	1.7	76.7	1.7	68.3	0.0	83.3	1.7	98.3	8.3	100.0	8.3
Backbone	20	100.0	63.3	100.0	96.7	100.0	96.7	100.0	100.0	100.0	98.3	100.0	96.7	100.0	96.7
	50	98.3	71.7	98.3	83.3	98.3	85.0	93.3	80.0	98.3	76.7	100.0	76.7	100.0	98.3
	75	96.7	51.7	90.0	36.7	96.7	58.3	73.3	38.3	96.7	65.0	100.0	75.0	98.3	88.3
	100	100.0	50.0	83.3	15.0	81.7	33.3	53.3	11.7	93.3	50.0	98.3	73.3	100.0	83.3
MonoBridge	10	100.0	55.0	100.0	98.3	100.0	98.3	100.0	98.3	100.0	90.0	100.0	100.0	100.0	100.0
	20	100.0	36.7	100.0	96.7	100.0	93.3	100.0	96.7	98.3	86.7	100.0	100.0	100.0	100.0
	50	98.3	13.3	96.7	66.7	93.3	71.7	68.3	26.7	96.7	58.3	100.0	61.7	100.0	83.3
	75	100.0	13.3	95.0	18.3	75.0	35.0	51.7	11.7	68.3	28.3	93.3	60.0	100.0	68.3
	100	100.0	1.7	88.3	1.7	80.0	20.0	48.3	0.0	83.3	13.3	91.7	23.3	100.0	53.3
	150	96.7	3.3	80.0	1.7	83.3	1.7	58.3	0.0	78.3	1.7	93.3	21.7	90.0	31.7
	200	98.3	0.0	66.7	0.0	78.3	0.0	65.0	0.0	83.3	0.0	93.3	16.7	73.3	23.3

Table 1. Satisfiability decision accuracy (Sat., %) and witness validity (Wit., %, SAT only). Each cell averages 10 formulas $\times$ 6 verbalizations. Difficulty at 0.5. Best in bold, second in italics.

Validation bias and variable assignment. Models can obtain deceptively high decision accuracy by over-predicting SAT. To separate decision priors from constructive reasoning, we additionally require a verifiable SAT witness: when predicting SAT, the model must output an assignment that satisfies all clauses under deterministic checking. We therefore report (i) SAT/UNSAT decision accuracy (Sat.), and (ii) witness validity (Wit.) on SAT instances. To quantify decision bias, we also report SAT prediction rate on UNSAT instances (Appendix Table 5). Each setting contains 10 independently generated formulas; the 6 verbalizations per formula are treated as repeated measurements and aggregated per formula for significance testing. The full per-model SAT/UNSAT confusion matrix is reported in Section M.

Generator difficulty at comparable formula size. With $|C|$ and difficulty fixed (0.5), ImplicationCycle is consistently harder than the SAT generators: UNSAT requires finding a contradiction certificate (reachability $x \rightsquigarrow \neg x$ and $\neg x \rightsquigarrow x$) that depends on long-range implication integration, whereas EquivalenceCore and Backbone often allow local propagation over repeated equivalence patterns. Llama-3.3-70B-Instruct also shows a strong SAT bias, predicting SAT on 88.3% of UNSAT instances (Appendix Table 5). At $|C| = 50$, mean accuracy suggests ImplicationCycle and EquivalenceCore are hardest, then Backbone, with MonoBridge easiest.

5.1. Impact of Difficulty Parameters

To assess whether generator parameters matter beyond $|C|$, we sweep each parameter over a 7-point grid

0, 0.2, 1/3, 0.5, 2/3, 0.8, 1 while fixing the others at 0.5. Each formula is one sample (10 per setting), with verbalizations aggregated per formula. We apply Friedman’s test across levels and Benjamini–Hochberg correction within each ablation family. Sweeps are run at clause counts near each generator’s transition region: $|C| = 75$ for ImplicationCycle, MonoBridge, and Backbone, and $|C| = 20$ for EquivalenceCore. Appendix Table 6 shows that several effects are significant only under the 7-point sweep or only after aggregation across sizes. We therefore interpret these trends as targeted brittleness regimes rather than universal monotonic effects.

MonoBridge parameters. The positive-part ratio has a significant effect ($p < 0.05$ at $|C| = 75$): instances are hardest near a balanced split (ratio ≈ 0.5) and easier when one monotone region dominates, consistent with a simple majority-assignment heuristic in the imbalanced regime and the need to reason about the cross-polarity bridge when balanced. Variable density also has a strong effect ($p \ll 0.001$): low density favors simplification via repeated variables, intermediate density produces a performance valley, and some large models recover at high density as the bridge becomes more salient. Qwen3-Next-80B and GPT-OSS-120B show improved performance around density 0.6 to 0.8, which is consistent with the bridge becoming more salient. The bridge position is not significant under our current protocol. One possible explanation is that many models do not process clauses strictly sequentially: they often restate the problem as an unordered set of constraints before attempting simplification.

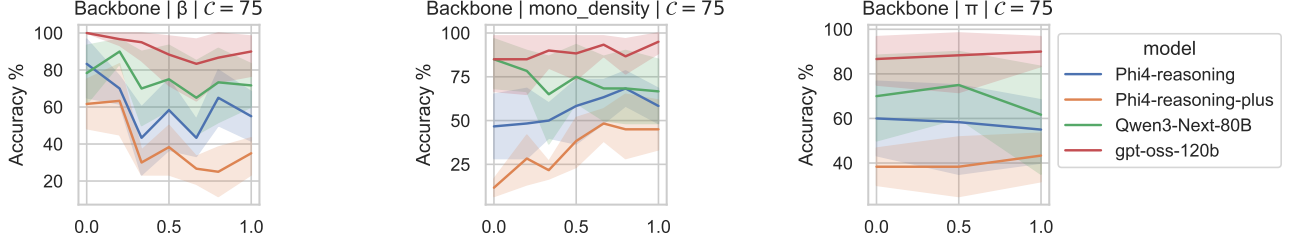


Figure 1. Main three difficulty parameters for Backbone at $|C| = 75$. Shade corresponds to 95% confidence interval. For π , $\pi = 0$ for a negative and $\pi = 1$ for a positive monotone part, while at $\pi = 0.5$ the sign is randomly chosen.

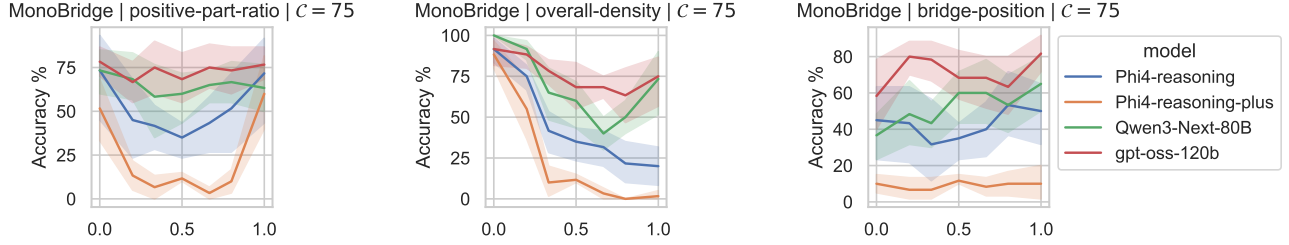


Figure 2. Main three difficulty parameters for MonoBridge at $|C| = 75$. Shade corresponds to 95% confidence interval.

Backbone parameters. The three difficulty parameters identified for Backbone have a limited impact on the performance compared to, for instance, the number of clauses. First, the *backbone fraction* β has the most significant effect on the performance (p -value < 0.005 for $|C| = 75$ and p -value < 0.02 overall), but this effect is marginal compared to the effect of the difficulty parameters of the other generators, with a drop of around 10% in the average performance between $\beta = 0$ and $\beta = 1$. This indicates that the backbone part, which has pairs of related clauses, is more complex to handle than the monotone part. While the *density of the monotone part* (mono density) has the next most significant impact on performance, it is significant only when considering $|C| = 75$ where we observe an increase in performance of up to 20% for the worst model. Higher monotone density means fewer distinct literals and stronger propagation. Finally, the polarity π of the monotone part has no significant impact on the performance, with models manipulating literal polarity without issues in the CoT traces we analyzed (all p -values > 0.9).

EquivalenceCore parameter. The free-to-bound variable ratio has a strong and consistent effect (Figure 3), with significant effects both overall and in the 7-point sweep at $|C| = 20$. (p -value < 0.005 for $|C| = 20$ and p -value $= 0.31$ overall). High values of the ratio, corresponding to low propagation, are significantly harder to solve than low ratios, as the models rely on early simplification of equivalence classes through propagation. Across CoT traces, all models managed to properly identify equivalences from the 2-CNF in its construction order.

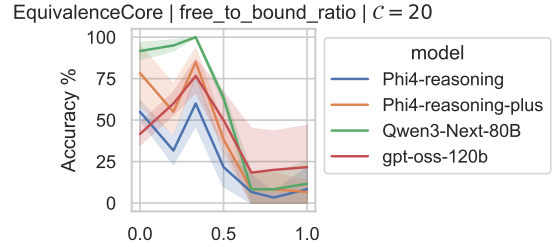


Figure 3. Main difficulty parameter for EquivalenceCore at $|C| = 20$. Shade corresponds to 95% confidence interval.

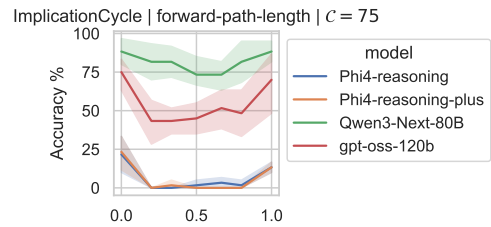


Figure 4. Main difficulty parameter for implication cycle at $|C| = 75$. Shade corresponds to 95% confidence interval.

ImplicationCycle parameter. The ImplicationCycle split parameter (k/m) affects performance mainly at larger sizes: extreme splits create one short side of the contradiction cycle, offering a shorter certificate path to track, while near-balanced splits force long-range reachability reasoning in both directions. The effect is significant at $|C| = 75$ but does not consistently generalize across sizes, so we see it as a targeted brittleness regime rather than a universal trend.

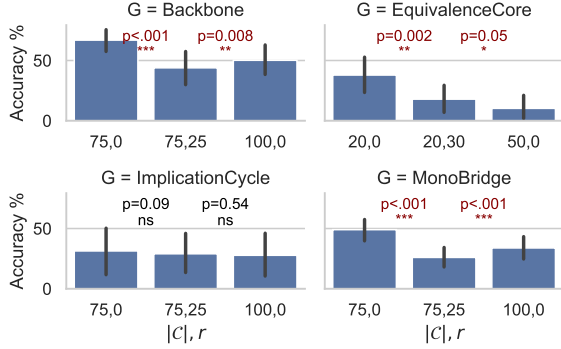


Figure 5. Effect of adding non-core filler clauses. We compare fixed-core settings with and without filler, and matched-total-size settings where a smaller core plus filler is compared to a larger core alone. Error bars are 95% confidence intervals across models and verbalization templates.

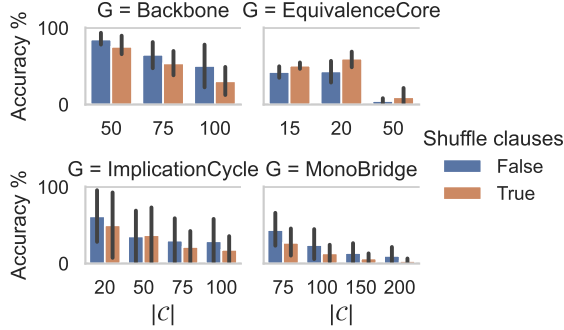


Figure 6. Comparison of accuracy between the construction order and a shuffling of the clauses. All difficulty parameters are set at 0.5. Error bars are 95% confidence intervals. For UNSAT problems, accuracy is measured on the prediction of satisfiability, and on variable assignment for SAT problems.

5.2. Impact of Non-core Variables

To test sensitivity to variables irrelevant to satisfiability, we add *filler* clauses over fresh variables. Let V_{fresh} be disjoint from the core variable set and define literals $\ell_i \in v_i, \neg v_i$ for $v_i \in V_{\text{fresh}}$. We sample filler clauses from pairs of these literals, choosing $|V_{\text{fresh}}|$ so filler and core clauses have comparable variable density. We use two comparison regimes: fixed core size and matched total clause count. Concretely, we compare $(|C| = 75, r = 25)$ to $(75, 0)$ and $(100, 0)$, and $(|C| = 20, r = 30)$ to $(20, 0)$ and $(50, 0)$. At difficulty 0.5, trends are visible but rarely significant, so we report results on the 3-level grid from Section 5.1 (0.2, 0.5, 0.8). For all generators except ImplicationCycle, adjacent settings differ significantly (paired t -tests; see Figure 5). Filler clauses significantly hurt EquivalenceCore, Backbone, and MonoBridge when the core size is fixed. Under matched total clauses, EquivalenceCore behaves dif-

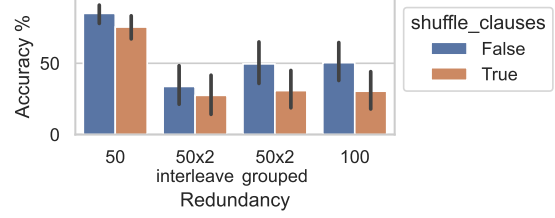


Figure 7. Comparison of accuracy when duplicating the clauses. All difficulty parameters are set at 0.5. Error bars are 95% confidence intervals. For UNSAT, accuracy is measured on the prediction of satisfiability, and on variable assignment for SAT problems.

ferently: a smaller equivalence core plus filler can be easier than a larger core alone. Trace inspection suggests models prioritize equivalences, but when they fail to compress equivalence classes, they repeatedly re-verify local consistency, making equivalences more costly than filler. For Backbone and MonoBridge, filler mainly adds noise, since their monotone regions already resemble filler structure, which further obscures the global pattern.

5.3. Sensitivity to Clause Order

We compare in Figure 6 our four main models under two clause orders: (i) the generator construction order (see Section 3) and (ii) a random permutation. Clause order significantly affects performance for all generators (paired t -tests, $p < 0.003$), with a weaker but still significant effect for ImplicationCycle ($p = 0.035$). For EquivalenceCore, shuffling is beneficial. As discussed in Section 5.2, when implication structure is poorly formalized, contiguous implication chains can act as misleading local patterns; shuffling breaks these groupings by separating related clauses, reducing early commitment to an incorrect structure.

5.4. Ability to Handle Repeated Patterns

Figure 7 shows results for Backbone at medium difficulty (all parameters set to 0.5) under clause repetition. We duplicate a base set of 50 clauses to obtain 100 clauses, using a fresh variable-to-entity mapping for the copy. We test two layouts: *grouped*, where the copy forms the last 50 clauses, and *interleave*, where copied clauses are randomly interleaved while preserving their internal order. We also compare against random clause permutations. Performance at 50 clauses differs significantly from all other variants (independent t -tests, $p < 0.001$). Shuffling significantly degrades performance within each setting (paired t -tests, $p < 0.001$), except for 50×2 *interleave* ($p = 0.036$). Differences between the 50×2 and 100-distinct-clauses conditions are not significant ($p > 0.1$), suggesting that repeating clauses is about as hard as adding distinct ones. Reasoning traces indicate that models rarely detect or exploit repeated patterns.

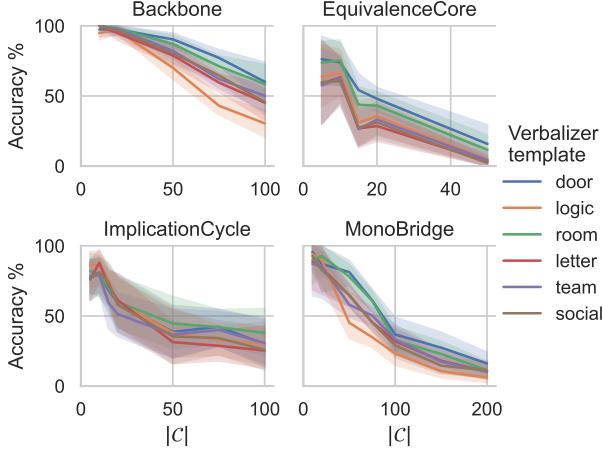


Figure 8. Decision accuracy on UNSAT and SAT witness validity against the number of clauses for each generator, depending on the verbalization template used. Shade is the 95% confidence interval.

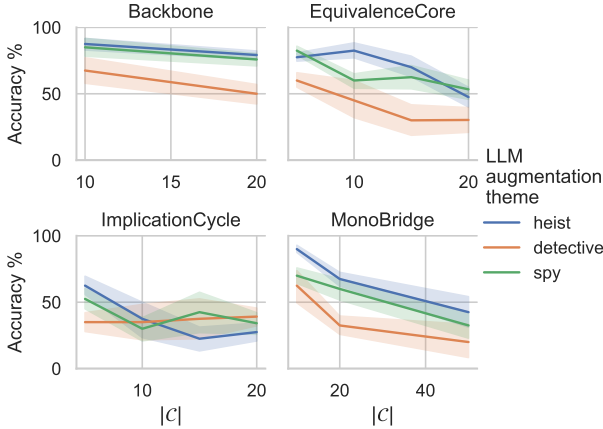


Figure 9. Decision accuracy on UNSAT and SAT witness validity against number of clauses for each generator, depending on the used LLM verbalization. Error bars are 95% confidence intervals.

5.5. Verbalization strategy

As shown in Figure 8, template-only verbalizations yield similar performance across templates, indicating little sensitivity to surface wording when the clause structure is explicit. In contrast, LLM verbalizer substantially degrades accuracy (Figure 9): at comparable clause sizes, performance drops by about 25 points relative to Figure 8. We attribute this mainly to increased contextual noise and weaker structural cues, which make clause extraction harder even though the underlying 2-CNF is information-preserving and can be recovered when models are explicitly prompted to do so. Across themes, detective narratives are hardest, followed by spy, then heist, consistent with the intended stylistic bias. Detective stories tend to be more indirect, which further obscures the logical form. For ImplicationCycle, theme effects are smaller: the shared-literal chain induces more co-

herent narratives, improving verbalization consistency and partially mitigating theme differences. Overall, templates make the 2-CNF structure easy to recognize, whereas LLM narratives often impede structure recovery and lead models to miss global logical patterns. Section L reports mean prompt-token counts for each verbalizer. At a fixed $|C|$, the LLM verbalizer produces prompts roughly 4–10 $\times$ longer than the templates (Section L).

6. Conclusion

We introduced a diagnostic 2-SAT benchmark based on parameterized 2-CNF families with controllable implication-graph structure, together with semantics-preserving perturbations such as clause reordering, variable renaming, and non-core fillers. The benchmark is designed to test whether solver-like behavior remains stable when satisfiability is unchanged but presentation, distractor load, or witness structure is varied. Across several reasoning models, these targeted interventions induce sharp performance shifts even at fixed instance size. Long-cycle UNSAT instances are particularly brittle, and the gap between decision accuracy and assignment validity shows that correct SAT/UNSAT predictions can mask weak witness construction. Sensitivity to ordering, fillers, and repetition further indicates limited invariance and limited reuse of intermediate structure. The restriction to 2-SAT is deliberate. Its implication graph and SCC decomposition provide a fine-grained and interpretable mapping from graph-theoretic properties, such as cycle length, backbone fraction, and bridge placement, to controlled diagnostic probes. Extending our work to richer constraint classes, while preserving this separation between structure and search hardness, is a natural direction for future work.

Acknowledgments

This work was supported by ANR-22-CE23-0002 ERI-ANA and ANR-25-CE23-7498 DesiRes, and by access to the HPC resources of IDRIS under the allocation 2026-AD011013338 made by GENCI.

Impact Statement

This paper aims to advance the understanding of reasoning capabilities and failure modes in LRMs through controlled diagnostic evaluation on logical satisfiability problems. We do not foresee direct negative societal consequences from this work. By identifying brittleness regimes that are not captured by aggregate accuracy metrics, the proposed benchmark may support more careful evaluation of LLM-based reasoners, especially in applications where logical consistency and verifiable outputs are important.

References

- Ariyani, N. F., Bouraoui, Z., Booth, R., and Schockaert, S. There’s no such thing as simple reasoning for llms. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T. (eds.), *Findings of the Association for Computational Linguistics, ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pp. 4503–4514. Association for Computational Linguistics, 2025. URL <https://aclanthology.org/2025.findings-acl.232/>.
- Bubeck, S., Chandrasekaran, V., Eldan, R., Gehrke, J., Horvitz, E., Kamar, E., Lee, P., Lee, Y. T., Li, Y., Lundberg, S., Nori, H., Palangi, H., Ribeiro, M. T., and Zhang, Y. Sparks of artificial general intelligence: Early experiments with gpt-4, 2023. URL <https://arxiv.org/abs/2303.12712>.
- Clark, P., Tafjord, O., and Richardson, K. Transformers as soft reasoners over language. In Bessiere, C. (ed.), *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020*, pp. 3882–3890. ijcai.org, 2020. doi: 10.24963/ijcai.2020/537. URL <https://doi.org/10.24963/ijcai.2020/537>.
- Creswell, A., Shanahan, M., and Higgins, I. Selection-inference: Exploiting large language models for interpretable logical reasoning. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. URL <https://openreview.net/forum?id=3Pf3Wg6o-A4>.
- Das, A., Khalid, I., Peñaloza, R., and Schockaert, S. When no paths lead to Rome: Benchmarking systematic neural relational reasoning. In *Proceedings of The Thirty-Ninth Annual Conference on Neural Information Processing Systems*, 2025.
- Giadikiaroglou, P., Lymperaioi, M., Filandrianos, G., and Stamou, G. Puzzle solving using reasoning of large language models: A survey. In Al-Onaizan, Y., Bansal, M., and Chen, Y. (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, pp. 11574–11591. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.EMNLP-MAIN.646. URL <https://doi.org/10.18653/v1/2024.emnlp-main.646>.
- Han, S., Schoelkopf, H., Zhao, Y., Qi, Z., Riddell, M., Zhou, W., Coady, J., Peng, D., Qiao, Y., Benson, L., Sun, L., Wardle-Solano, A., Szabó, H., Zubova, E., Burtell, M., Fan, J., Liu, Y., Wong, B., Sailor, M., Ni, A., Nan, L., Kasai, J., Yu, T., Zhang, R., Fabbri, A. R., Kryscinski, W., Yavuz, S., Liu, Y., Lin, X. V., Joty, S., Zhou, Y., Xiong, C., Ying, R., Cohan, A., and Radev, D. FOLIO: natural language reasoning with first-order logic. In Al-Onaizan, Y., Bansal, M., and Chen, Y. (eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, pp. 22017–22031, 2024a. URL <https://dblp.org/rec/conf/emnlp/HanS0QRZCPQBSWS24>.
- Han, S., Yu, A., Shen, R., Qi, Z., Riddell, M., Zhou, W., Qiao, Y., Zhao, Y., Yavuz, S., Liu, Y., Joty, S., Zhou, Y., Xiong, C., Radev, D., Ying, R., and Cohan, A. P-FOLIO: evaluating and improving logical reasoning with abundant human-written reasoning chains. In Al-Onaizan, Y., Bansal, M., and Chen, Y. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, Florida, USA, November 12-16, 2024*, pp. 16553–16565. Association for Computational Linguistics, 2024b. doi: 10.18653/V1/2024.FINDINGS-EMNLP.966. URL <https://doi.org/10.18653/v1/2024.findings-emnlp.966>.
- Hazra, R., Venturato, G., Dos Martires, P. Z., and De Raedt, L. Have large language models learned to reason? a characterization via 3-SAT. In *Second Conference on Language Modeling*, 2025.
- Heyman, A. and Zylberberg, J. Evaluating the systematic reasoning abilities of large language models through graph coloring. *CoRR*, abs/2502.07087, 2025. doi: 10.48550/ARXIV.2502.07087. URL <https://doi.org/10.48550/arXiv.2502.07087>.
- Jiang, D., Fonseca, M., and Cohen, S. LeanReasoner: Boosting complex logical reasoning with lean. In Duh, K., Gomez, H., and Bethard, S. (eds.), *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pp. 7497–7510, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.naacl-long.416. URL <https://aclanthology.org/2024.naacl-long.416/>.
- Kazemi, M., Yuan, Q., Bhatia, D., Kim, N., Xu, X., Imbrasaite, V., and Ramachandran, D. BoardgameQA: A dataset for natural language reasoning with contradictory information. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/7adce80e86aa841490e6307109094de5-Abstract-Datasets_and_Benchmarks.html.

- Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., and Iwasawa, Y. Large language models are zero-shot reasoners. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/8bb0d291acd4acf06ef112099c16f326-Abstract-Conference.html.
- Li, D., Cao, S., Griggs, T., Liu, S., Mo, X., Tang, E., Hegde, S., Hakhamaneshi, K., Patil, S. G., Zaharia, M., Gonzalez, J. E., and Stoica, I. LLMs can easily learn to reason from demonstrations structure, not content, is what matters! *CoRR*, abs/2502.07374, 2025. doi: 10.48550/ARXIV.2502.07374. URL <https://doi.org/10.48550/arXiv.2502.07374>.
- Lin, B. Y., Bras, R. L., Richardson, K., Sabharwal, A., Poovendran, R., Clark, P., and Choi, Y. Zeblogic: On the scaling limits of LLMs for logical reasoning. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=sTAJ9QyA6l>.
- Liu, J., Cui, L., Liu, H., Huang, D., Wang, Y., and Zhang, Y. LogiQA: A challenge dataset for machine reading comprehension with logical reasoning. In Bessiere, C. (ed.), *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI 2020*, pp. 3622–3628. ijcai.org, 2020. doi: 10.24963/IJCAI.2020/501. URL <https://doi.org/10.24963/ijcai.2020/501>.
- Pan, L., Ganesh, V., Abernethy, J., Esposito, C., and Lee, W. Can transformers reason logically? a study in SAT solving. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=5BGC2I2fxx>.
- Parmar, M., Patel, N., Varshney, N., Nakamura, M., Luo, M., Mashetty, S., Mitra, A., and Baral, C. LogicBench: Towards systematic evaluation of logical reasoning ability of large language models. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 13679–13707, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.739. URL <https://aclanthology.org/2024.acl-long.739/>.
- Patel, A., Li, B., Rasooli, M. S., Constant, N., Raffel, C., and Callison-Burch, C. Bidirectional language models are also few-shot learners. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. URL <https://openreview.net/forum?id=wCFB37bzud4>.
- Rameshkumar, R., Huang, J., Sun, Y., Xia, F., and Saparov, A. Reasoning models reason well, until they don’t. In *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, pp. 936–956, Mumbai, India, December 2025. The Asian Federation of Natural Language Processing and The Association for Computational Linguistics. doi: 10.18653/v1/2025.ijcnlp-long.52. URL <https://aclanthology.org/2025.ijcnlp-long.52/>.
- Richardson, K. and Sabharwal, A. Pushing the limits of rule reasoning in transformers through natural language satisfiability. In *Thirty-Sixth AAAI Conference on Artificial Intelligence, AAAI 2022, Thirty-Fourth Conference on Innovative Applications of Artificial Intelligence, IAAI 2022, The Twelveth Symposium on Educational Advances in Artificial Intelligence, EAAI 2022 Virtual Event, February 22 - March 1, 2022*, pp. 11209–11219. AAAI Press, 2022. doi: 10.1609/aaai.v36i10.21371. URL <https://doi.org/10.1609/aaai.v36i10.21371>.
- Saeed, M., Ahmadi, N., Nakov, P., and Papotti, P. Rule-BERT: Teaching soft rules to pre-trained language models. In Moens, M.-F., Huang, X., Specia, L., and Yih, S. W.-t. (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 1460–1476, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.110. URL <https://aclanthology.org/2021.emnlp-main.110/>.
- Saha, S., Ghosh, S., Srivastava, S., and Bansal, M. PRouter: Proof generation for interpretable reasoning over rules. In Webber, B., Cohn, T., He, Y., and Liu, Y. (eds.), *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 122–136, Online, November 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.emnlp-main.9. URL <https://aclanthology.org/2020.emnlp-main.9/>.
- Saparov, A. and He, H. Language models are greedy reasoners: A systematic formal analysis of chain-of-thought. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. URL <https://openreview.net/forum?id=qFVVBzXxR2V>.

- Shojaee, P., Mirzadeh, S. I., Alizadeh, K., Horton, M., Bengio, S., and Farajtabar, M. The illusion of thinking: Understanding the strengths and limitations of reasoning models via the lens of problem complexity. In *The Thirtieth Annual Conference on Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=YghiOusmvw>.
- Sinha, K., Sodhani, S., Dong, J., Pineau, J., and Hamilton, W. L. CLUTRR: A diagnostic benchmark for inductive reasoning from text. In Inui, K., Jiang, J., Ng, V., and Wan, X. (eds.), *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019*, pp. 4505–4514. Association for Computational Linguistics, 2019. doi: 10.18653/V1/D19-1458. URL <https://doi.org/10.18653/v1/D19-1458>.
- Tafjord, O., Dalvi, B., and Clark, P. ProofWriter: Generating implications, proofs, and abductive statements over natural language. In Zong, C., Xia, F., Li, W., and Navigli, R. (eds.), *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pp. 3621–3634, Online, August 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.findings-acl.317. URL <https://aclanthology.org/2021.findings-acl.317/>.
- Tian, J., Li, Y., Chen, W., Xiao, L., He, H., and Jin, Y. Diagnosing the first-order logical reasoning ability through LogicNLI. In Moens, M.-F., Huang, X., Specia, L., and Yih, S. W.-t. (eds.), *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp. 3738–3747, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.emnlp-main.303. URL <https://aclanthology.org/2021.emnlp-main.303/>.
- Valmeekam, K., Stechly, K., and Kambhampati, S. LLMs still can’t plan; can LRMs? A preliminary evaluation of OpenAI’s o1 on PlanBench. *CoRR*, abs/2409.13373, 2024. doi: 10.48550/ARXIV.2409.13373. URL <https://doi.org/10.48550/arXiv.2409.13373>.
- Valmeekam, K., Palod, V., Stechly, K., Gundawar, A., and Kambhampati, S. Beyond semantics: The unreasonable effectiveness of reasonless intermediate tokens. *Transactions on Machine Learning Research (TMLR)*, 2025a. URL <https://arxiv.org/abs/2505.13775>.
- Valmeekam, K., Stechly, K., Gundawar, A., and Kambhampati, S. A systematic evaluation of the planning and scheduling abilities of the reasoning model o1. *Trans. Mach. Learn. Res.*, 2025, 2025b. URL <https://openreview.net/forum?id=FkKBxp0FhR>.
- Wang, B., Min, S., Deng, X., Shen, J., Wu, Y., Zettlemoyer, L., and Sun, H. Towards understanding chain-of-thought prompting: An empirical study of what matters. In Rogers, A., Boyd-Graber, J., and Okazaki, N. (eds.), *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 2717–2739, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.153. URL <https://aclanthology.org/2023.acl-long.153/>.
- Wei, A., Wu, Y., Wan, Y., Suresh, T., Tan, H., Zhou, Z., Koyejo, S., Wang, K., and Aiken, A. SAT-Bench: Benchmarking LLMs’ logical reasoning via automated puzzle generation from SAT formulas. In Christodoulopoulos, C., Chakraborty, T., Rose, C., and Peng, V. (eds.), *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 33832–33849, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-332-6. doi: 10.18653/v1/2025.emnlp-main.1716. URL <https://aclanthology.org/2025.emnlp-main.1716/>.
- Yu, W., Jiang, Z., Dong, Y., and Feng, J. Reclor: A reading comprehension dataset requiring logical reasoning. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. URL <https://openreview.net/forum?id=HJgTtT4tvB>.
- Zhang, H., Li, L. H., Meng, T., Chang, K., and den Broeck, G. V. On the paradox of learning to reason from data. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence, IJCAI 2023, 19th-25th August 2023, Macao, SAR, China*, pp. 3365–3373. ijcai.org, 2023. doi: 10.24963/ijcai.2023/375. URL <https://doi.org/10.24963/ijcai.2023/375>.
- Zhao, Y., Li, Y., Bo, Z.-H., Takezoe, R., Hui, H., Guang, M., Ren, L., Qin, X., and Long, K. SATQuest: A verifier for logical reasoning evaluation and reinforcement finetuning of LLMs. In *ICLR 2026 Workshop on Logical Reasoning of Large Language Models*, 2026. URL <https://openreview.net/forum?id=Qc2lUJsgOt>.
- Zhu, Q., Huang, F., Peng, R., Lu, K., Yu, B., Cheng, Q., Qiu, X., Huang, X., and Lin, J. Autologi: Automated generation of logic puzzles for evaluating reasoning abilities of large language models. *CoRR*, abs/2502.16906, 2025. doi: 10.48550/ARXIV.2502.16906. URL <https://doi.org/10.48550/arXiv.2502.16906>.

A. Background

Let $\mathcal{V}$ be a finite set of variables.

A *literal* is either a variable p (positive) or its negation $\neg p$ (negative). Two literals are *complementary* if one is the negation of the other.

A *clause* is a disjunction of literals. A formula is in *Conjunctive Normal Form* (CNF) if it is a conjunction of clauses. A 2-CNF formula is a CNF formula where each clause contains at most two literals.

An *interpretation* is a function $\omega : \mathcal{V} \rightarrow \{0, 1\}$, where 1 stands for *true* and 0 for *false*. It is extended to formulas as follows: $\omega(\neg\varphi) = 1 - \omega(\varphi)$, $\omega(\varphi \wedge \psi) = \min(\omega(\varphi), \omega(\psi))$, and $\omega(\varphi \vee \psi) = \max(\omega(\varphi), \omega(\psi))$.

A model of a formula φ is an interpretation ω that satisfies the formula: $\omega(\varphi) = 1$. In particular, ω is a model of a CNF formula φ if and only if for each clause c in φ , there exists a literal l in c such that $\omega(l) = 1$.

A formula is *satisfiable* if and only if it has at least one model; otherwise, it is *unsatisfiable*.

Definition A.1 (Implication Graph). Let φ be a 2-CNF formula. The *implication graph* of φ is a directed graph $G = (V, E)$, where $V = \text{Lit}(\text{Var}(\varphi))$ and $E = \bigcup \{ \{(\bar{l}, l'), (\bar{l}', l)\} : l \vee l' \in \varphi \text{ or } (l \in \phi \text{ and } l' = l) \}$.

We use $l \rightsquigarrow l'$ to denote that there is a path from l to l' in the implication graph.

Definition A.2 (Proof Cycle). Let φ be an unsatisfiable 2-CNF formula. A *proof cycle* of the unsatisfiability of φ is a cycle $(l) \rightarrow (l_1) \rightarrow \dots \rightarrow (l_k) \rightarrow (l)$ in the implication graph of ϕ , where there exists $i \in \{1, \dots, k\}$ such that $l_i = \bar{l}$.

Theorem A.3. A 2-CNF formula is unsatisfiable iff it admits a proof cycle.

B. UNSAT Generator using Implication Cycles

We create one clause per implication shown above:

$$\begin{aligned} &(\neg \ell_1 \vee \ell_2), (\neg \ell_2 \vee \ell_3), \dots, (\neg \ell_{k-1} \vee \ell_k), (\neg \ell_k \vee \neg \ell_1), \\ &(\neg \neg \ell_1 \vee \ell_{k+1}), (\neg \ell_{k+1} \vee \ell_{k+2}), \dots, (\neg \ell_{m-1} \vee \ell_m), (\neg \ell_m \vee \ell_1). \end{aligned}$$

We describe our generator in Algorithm 1.

Example 1 (UNSAT core with $m=5, k=3$). Let $(\ell_1, \dots, \ell_5) = (v_1, \neg v_2, v_3, v_4, \neg v_5)$. Core clauses:

$$\begin{aligned} &(\neg v_1 \vee \neg v_2), (v_2 \vee v_3), (\neg v_3 \vee \neg v_1), \\ &(v_1 \vee v_4), (\neg v_4 \vee \neg v_5), (v_5 \vee v_1). \end{aligned}$$

This forces $v_1 \rightsquigarrow \neg v_1$ and $\neg v_1 \rightsquigarrow v_1$, hence UNSAT.

C. SAT Generator via Free Variables

For each $y \in \mathcal{V} \setminus F$:

$$\tau(y) = p(y) : (\neg y \vee p(y)) \wedge (\neg p(y) \vee y), \quad \tau(y) = \neg p(y) : (\neg y \vee \neg p(y)) \wedge (p(y) \vee y).$$

This generator is described in Algorithm 2.

Example 2. Let $F = \{p\}$ (free) and define $q \leftrightarrow \neg p, r \leftrightarrow p$. Clauses:

$$(\neg q \vee \neg p) \wedge (p \vee q) \wedge (\neg r \vee p) \wedge (\neg p \vee r).$$

There are exactly two models, extending $p \in \{0, 1\}$.

D. Backbone-based Method

Pseudocode:

Algorithm 1 GENERATEUNSAT2CNF($n, m, k, r, shuffle$)

Require: $n \geq m \geq 2, 1 \leq k \leq m, r \geq 0, shuffle \in \{\text{true}, \text{false}\}$

- 1: Create variables $\mathcal{V} = \{x_1, \dots, x_n\}$; pick distinct $v_1, \dots, v_m \in \mathcal{V}$
- 2: For $i = 1..m$, choose a polarity $\ell_i \in \{v_i, \neg v_i\}$
- 3: Initialize clause set $\mathcal{C} \leftarrow \emptyset$ {Forward part}
- 4: **for** $i = 1$ to $k - 1$ **do**
- 5: $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\neg \ell_i \vee \ell_{i+1})\}$
- 6: **end for**
- 7: $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\neg \ell_k \vee \neg \ell_1)\}$ {Backward part}
- 8: $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\neg \neg \ell_1 \vee \ell_{k+1})\}$
- 9: **for** $i = k + 1$ to $m - 1$ **do**
- 10: $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\neg \ell_i \vee \ell_{i+1})\}$
- 11: **end for**
- 12: $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\neg \ell_m \vee \ell_1)\}$ {Fillers}
- 13: **for** $j = 1$ to r **do**
- 14: Add a filler clause $(\lambda \vee \lambda')$ over $\mathcal{V}$ (optionally avoid tautologies)
- 15: **end for**
- 16: **if** $shuffle$ **then**
- 17: Randomly permute $\mathcal{C}$
- 18: **end if**
- 19: Output $\Phi = \bigwedge_{C \in \mathcal{C}} C = 0$

Algorithm 2 GENERATESAT2CNF(n, f)

Require: $n \geq 1, 0 \leq f \leq n$

- 1: Create variables $\mathcal{V} = \{x_1, \dots, x_n\}$; choose free set $F \subseteq \mathcal{V}$ with $|F| = f$
- 2: $\mathcal{C} \leftarrow \emptyset$
- 3: **for each** $y \in \mathcal{V} \setminus F$ **do**
- 4: Pick $p(y) \in F$ and sign $s \in \{+1, -1\}$
- 5: **if** $s = +1$ **then**
- 6: $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\neg y \vee p(y)), (\neg p(y) \vee y)\}$
- 7: **else**
- 8: $\mathcal{C} \leftarrow \mathcal{C} \cup \{(\neg y \vee \neg p(y)), (p(y) \vee y)\}$
- 9: **end if**
- 10: **end for**
- 11: Output $\Phi = \bigwedge_{C \in \mathcal{C}} C = 0$

1. Input: $n, B \subseteq \mathcal{V}$, planted b , orientation $\pi \in \{+, -\}$, m_{mono} .
2. $F \leftarrow \mathcal{V} \setminus B$; initialize clause multiset $\mathcal{C} \leftarrow \emptyset$.
3. For each $x \in B$: introduce a_x and add $(x \vee a_x), (x \vee \neg a_x)$ if $b(x) = 1$, else $(\neg x \vee a_x), (\neg x \vee \neg a_x)$.
4. Repeat m_{mono} times: sample $y, z \in F, y \neq z$, and add $(\neg y \vee \neg z)$ if $\pi = -$, else $(y \vee z)$.
5. Output $\Phi^* = \bigwedge_{C \in \mathcal{C}} C$.

E. Monotone Split with a Moving Cross-Polarity Bridge

Construction:

1. **Disjoint variable sets.** Partition the variables into $P = \{p_1, \dots, p_{n_+}\}$ and $N = \{n_1, \dots, n_{n_-}\}$ with $P \cap N = \emptyset$.
2. **Positive-monotone core Φ_+ .** Generate m_+ clauses, each $(p_i \vee p_j)$ for some $p_i, p_j \in P$ (avoid tautologies and duplicates if desired).
3. **Negative-monotone core Φ_- .** Generate m_- clauses, each $(\neg n_i \vee \neg n_j)$ for some $n_i, n_j \in N$.
4. **Bridge clause B .** Pick endpoints $p^* \in P$ and $n^* \in N$ and set

$$B := (\neg p^* \vee n^*).$$

5. **Ordering (moving the bridge).** Fix an overall ordering of $\Phi_+ \cup \Phi_-$ (e.g., all Φ_+ then all Φ_- or any shuffle). Let $M = m_+ + m_-$ be the number of monotone clauses. Choose an index $s \in \{1, \dots, M+1\}$ and *insert B as the s -th clause*. Varying s across instances moves where the solver first encounters the coupling.

Proposition E.1 (Satisfiable by construction). *The formula $\Phi := \Phi_+ \wedge \Phi_- \wedge B$ is satisfiable.*

For example, the assignment that set all P to 1 and set n^* to 1 and all other $N \setminus \{n^*\}$ to 0 satisfies Φ_+ (positives are 1), satisfies B (since $n^* = 1$), and satisfies every $(\neg n_i \vee \neg n_j)$ in Φ_- (at least one of n_i, n_j is 0).

Backtracking trigger. If a solver's early decisions tend to make $p^* = 1$ (natural for Φ_+) and $n^* = 0$ (natural for Φ_-), then when B is finally processed it becomes $(\neg 1 \vee 0)$ and conflicts, forcing a revision (unit propagation or backtrack). By changing the insertion index s , we control when this conflict becomes visible.

Pseudocode

1. Input: $n_+, n_-, m_+, m_-, p^* \in P, n^* \in N$, insertion index $s \in [1, M+1]$.
2. Build lists C_+ of m_+ clauses $(p \vee p')$ over P and C_- of m_- clauses $(\neg n \vee \neg n')$ over N .
3. Concatenate $C := \text{order}(C_+, C_-)$ (any fixed order or shuffle); let $M = |C|$.
4. Insert $B = (\neg p^* \vee n^*)$ at position s in C .
5. Output $\Phi = \bigwedge_{i=1}^{M+1} C[i]$.

F. Symmetry/Redundancy Probe

Construction

1. **Base formula.** Let $\Psi = \bigwedge_{c \in \mathcal{C}} c$ be any 2-CNF over variables $V = \{x_1, \dots, x_n\}$.
2. **Disjoint renaming.** Choose a bijection $\rho : V \rightarrow V' = \{x'_1, \dots, x'_n\}$ with $V \cap V' = \emptyset$. Extend ρ to literals and clauses in the obvious way (e.g., $\rho(\neg x) = \neg \rho(x)$, $\rho(\ell_1 \vee \ell_2) = \rho(\ell_1) \vee \rho(\ell_2)$).
3. **Concatenation.** Define

$$\Phi := \Psi \wedge \rho(\Psi).$$

Optionally, order the clauses as (i) all of Ψ then all of $\rho(\Psi)$ (grouped), or (ii) an interleaving of the two clause lists (to blur copy boundaries).

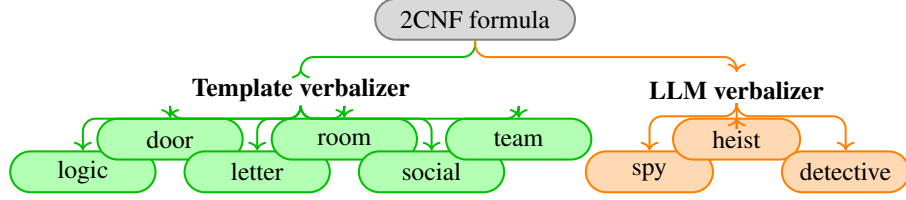


Figure 10. Summary of the verbalization process

G. Verbalization Templates and Prompts

G.1. Template Verbalizers

Each template verbalizer converts a 2-CNF clause ($\ell_1 \vee \ell_2$) into a natural language sentence following a fixed pattern. The verbalizer also provides a context preamble and a final yes/no question. Table 2 summarizes the six schemes.

Scheme	True value	False value	Entity naming	Context preamble
logic	true	false	Letters: A, B, ..., Z, AA, AB, ...	Consider these logical relationships:
letter	has	doesn't have	Person names	Consider these rules about who has the letter:
team	Red	Blue	Person names	We have team assignment rules:
social	attends	doesn't attend	Person names	You're planning a party with these attendance conditions:
room	lit	dark	room 1, room 2, ...	A building has lighting rules:
door	open	closed	door 1, door 2, ...	A facility has door coupling rules:

Table 2. Summary of the six template verbalizers. Each row shows the scheme name, the context preamble, the true/false value labels, and the entity naming convention.

Clause-to-text patterns. Below are the exact sentence templates used for each scheme. In all cases, A and B denote the entity names of the two literals' variables, and the polarity determines which value (true or false) is used.

- **logic:** Either {A} is true or {B} is false (or both).
- **letter:** Either {A} has the letter or {B} doesn't have the letter (or both).
- **team:** Either {A} is on the Red team or {B} is on the Blue team.
- **social:** {A} attends or {B} doesn't attend (or both).
- **room:** room {i} is lit or room {j} is dark (or both).
- **door:** door {i} is open or door {j} is closed (or both).

Final questions.

- **logic:** Can you assign truth values to all statements without creating a contradiction?
- **letter:** Is there a consistent way for people to hold or not hold the letter?
- **team:** Is there a valid team assignment that satisfies all constraints?
- **social:** Can you create a guest list that respects everyone's conditions?
- **room:** Can all the lighting rules be satisfied simultaneously?
- **door:** Is there a configuration of doors that satisfies all the rules?

Assembled output. For each formula, the verbalizer concatenates the context preamble, one sentence per clause, a blank line, and the final question. For example, with the `logic` scheme and clauses $(\neg A \vee B)$, $(A \vee \neg C)$:

Consider these logical relationships:

Either A is false or B is true (or both).

Either A is true or C is false (or both).

Can you assign truth values to all statements
without creating a contradiction?

G.2. Evaluation Prompt

The evaluation prompt is appended after the verbalized formula. Two variants exist depending on whether template or LLM verbalization is used.

Template verbalizer prompt.

{verbalized formula text}

Think step by step.

If a valid assignment exists, provide it as JSON
using "{true_val}" or "{false_val}" as values.

Example format:

```
{
  "{entity_1}": "{true_val}",
  "{entity_2}": "{false_val}",
  "{entity_3}": "{true_val}"
}
```

End your response with exactly one of:

- "The answer is: Yes" (if a valid assignment exists)
- "The answer is: No" (if no valid assignment exists)

where {true_val} and {false_val} are the scheme-specific values from Table 2, and the example JSON shows the first three entities.

LLM verbalizer prompt.

{augmented story text}

Think step by step.

If a valid assignment exists, provide it as JSON with one of the allowed values for each entity.

Entities and their possible values:

- "{entity_name}": "{true_desc}" or "{false_desc}"
- ...

Example JSON format:

```
{
  "entity_name": "value"
}
```

End your response with exactly one of:

- "The answer is: Yes" (if a valid assignment exists)
- "The answer is: No" (if no valid assignment exists)

where each entity is listed with its theme-specific true/false descriptions.

G.3. LLM Story Generation Prompts

The LLM verbalizer uses a multi-turn conversation to generate narrative clues. Below are the exact prompts used at each step.

Step 1: Global context generation.

You are a mystery story writer. Create a short introduction for a detective story.

CHARACTERS/OBJECTS in this story:

- {entity_1_name}
- {entity_2_name}
- ...

Requirements:

- Set the scene for a mystery with clues to discover
- Briefly introduce each character/object naturally in the narrative
- Do NOT mention any specific rules or relationships yet
- Keep it brief and atmospheric

Write only the introduction paragraph.

Step 2: Per-clause narrative generation. For each clause $(\ell_1 \vee \ell_2)$, the following prompt is appended to the conversation:

Write clue {i}/{total} for our mystery story.

This clue must express that AT LEAST ONE of these statements is true:

- Statement A: {entity_1_name} {state_1}
- Statement B: {entity_2_name} {state_2}

(It's possible both are true, but at least one MUST be true.)

Write 2-3 sentences that naturally convey this logical rule through detective reasoning, witness testimony, or evidence analysis. Be creative but ensure the logic is clear.

Write ONLY the clue paragraph:

Step 3a: Entity extraction (validation).

From this clue, identify the TWO main entities being discussed.

CLUE: "{generated_paragraph}"

AVAILABLE ENTITIES:

["entity_1", "entity_2", ...]

Which two entities does this clue primarily discuss?

Respond with ONLY a JSON array of exactly 2 entity names: ["entity1", "entity2"]

Model	Model ID	Param.	Max Tokens	License
Phi 4 reasoning	microsoft/Phi-4-reasoning	15B	32K	MIT
Phi 4 reasoning plus	microsoft/Phi-4-reasoning-plus	15B	32K	MIT
Qwen3-Next 80B	Qwen/Qwen3-Next-80B-A3B-Thinking	81B	262K	Apache 2.0
GPT-OSS-120B	openai/gpt-oss-120b	120B	131K	Apache 2.0
OLMo-3 32B	allenai/Olmo-3.1-32B-Think	32B	66K	Apache 2.0
QwQ 32B	Qwen/QwQ-32B	33B	41K	Apache 2.0
Llama-3.3 70B-Instruct	meta-llama/Llama-3.3-70B-Instruct	71B	66K	Llama 3.3

Table 3. Overview of the reasoning models.

Model	Max Tokens	Truncation Rate (%)
Phi-4-reasoning	16 384	14.9
Phi-4-reasoning-plus	16 384	30.5
Qwen3-Next 80B	32 768	1.3
GPT-OSS-120B	32 768	0.0
OLMo-3 32B	32 768	3.0
QwQ 32B	32 768	3.6
Llama-3.3 70B-Instruct	32 768	0.0

Table 4. Truncation rates by model under fixed output budget.

Step 3b: State extraction (validation). For each of the two extracted entities:

In this clue, what is stated about {entity_name}?

CLUE: "{generated_paragraph}"

The clue says that {entity_name}:

- A) {true_description}
- B) {false_description}

Which state does the clue indicate?

Answer with ONLY "A" or "B".

If validation fails after 3 attempts, a programmatic fallback sentence is used:

Either {entity_1} {state_1}, or {entity_2} {state_2} (or both).

H. Model Information

Table 3 reports general model information.

I. Truncation Rate

Table 4 reports output truncation rates per model.

J. SAT Prediction Bias on UNSAT Instances

Table 5 reports the proportion of UNSAT instances for which each model incorrectly predicted SAT. A high rate indicates a tendency to over-predict satisfiability regardless of the actual logical structure. Llama-3.3-70B-Instruct exhibits a particularly strong SAT bias (88.3%), suggesting it defaults to predicting satisfiability rather than performing genuine constraint reasoning.

Model	SAT Prediction (%)
Phi-4-reasoning	33.3
Phi-4-reasoning-plus	20.2
Qwen3-Next 80B	10.5
GPT-OSS-120B	26.4
OLMo-3 32B	21.7
QwQ 32B	34.0
Llama-3.3 70B-Instruct	88.3

Table 5. SAT prediction rate on UNSAT instances (ImplicationCycle generator). Higher values indicate stronger bias toward predicting satisfiability.

K. Statistical Significance Results

The significance results for Friedman’s test for repeated measurements across difficulty parameter levels with Benjamini–Hochberg correction are reported in Table 6.

L. Estimated Mean Token Count per Verbalizer

Table 7 reports the estimated mean token count (computed as $\lfloor \text{chars}/4 \rfloor$ on the rendered prompts) as a function of the number of clauses $|C|$, for the six template verbalizers used in the paper. Token count grows nearly linearly with $|C|$ in every verbalizer; the `letter` verbalizer is consistently the longest (about 1.5× the most compact), while `logic` and `room` are the shortest. The spread between verbalizers at fixed $|C|$ is bounded by a factor of ~ 1.6 , while the gap between *template* and *LLM-generated* verbalizations (multi-sentence paragraphs per clause) is substantially larger and explains part of the drop reported in Section 5.5.

Template vs. LLM-generated verbalization

The LLM verbalizer (see Section G) renders each clause as a multi-sentence paragraph, which yields substantially longer prompts than the template verbalizers at matched $|C|$. Table 8 reports the actual mean prompt-token count measured on the GPT-OSS-120B run, comparing the matched (generator, $|C|$) pairs available in both normal and augmented configurations. This systematic increase in prompt length partly explains the accuracy drop reported for the LLM verbalizer in Section 5.5, distinct from the effect of surface wording.

M. Per-Model Confusion Matrix

Table 9 reports the per-model confusion matrix for the SAT vs. UNSAT decision, aggregated over all generators and parameter settings of the main evaluation (Table 1). Convention: *Positive* = SAT. Llama-3.3-70B-Instruct stands out with a large false-positive count (FP = 371), consistent with the strong SAT-prediction bias already reported in Table 5. Qwen3-Next-80B achieves the highest accuracy (96.3%) and the most balanced error profile.

N. Proprietary Reasoning Models

We complement the open-weight evaluation of Table 1 with three recent proprietary reasoning models—o3-mini, o4-mini, and GPT-5.4-mini—run on the same prompts and the same protocol (10 formulas $\times$ 6 template verbalizers per cell). Table 10 reports per-(generator, $|C|$) decision accuracy (Sat., %) and witness validity (Wit., %, SAT instances only).

O. GPT-OSS-120B: Effect of Reasoning Effort

GPT-OSS exposes three reasoning-effort levels (`low`, `medium`, `high`) through the `reasoning_effort` field of its chat template when served locally via vLLM. The results reported in the main paper (Table 1) use the default setting (`medium`). To characterise how the compute budget allocated to internal reasoning interacts with the difficulty axes of our benchmark, we report in Table 11 the Sat. and Wit. at all three effort levels.

Generator	Difficulty parameter	$ \mathcal{C} $	p -value
Backbone	monotone density	Overall	$3.363e^{-3}$
		20	$4.724e^{-1}$
		50	$3.813e^{-2}$
		75	$4.724e^{-1}$
		75 7-point	$3.125e^{-1}$
		100	$3.877e^{-2}$
	β	Overall	$1.865e^{-2}$
		20	$1.462e^{-1}$
		50	$5.971e^{-2}$
		75	$3.877e^{-2}$
		75 7-point	$4.310e^{-3}$
		100	$3.679e^{-1}$
	π	75 7-point	$9.355e^{-1}$
EquivalenceCore	Free-to-bound variable ratio	Overall	$2.754e^{-5}$
		15	$1.832e^{-2}$
		20	$1.832e^{-2}$
		20 7-point	$1.296e^{-3}$
		50	$8.208e^{-2}$
ImplicationCycle	Forward path relative length	Overall	$8.116e^{-1}$
		5	$7.643e^{-2}$
		10	$5.488e^{-1}$
		15	$3.679e^{-1}$
		20	$2.574e^{-1}$
		50	$4.412e^{-1}$
		75	$2.725e^{-1}$
		75 7-point	$7.257e^{-3}$
		100	$5.292e^{-1}$
MonoBridge	Bridge relative position	Overall	$1.869e^{-1}$
		50	$1.889e^{-1}$
		75	$7.788e^{-1}$
		75 7-point	$2.154e^{-1}$
		100	$1.738e^{-1}$
		150	$1.496e^{-1}$
		200	$1.353e^{-1}$
	Overall variable density	Overall	$2.073e^{-7}$
		50	$4.979e^{-2}$
		75	$1.832e^{-2}$
		75 7-point	$2.719e^{-3}$
		100	$3.813e^{-2}$
		150	$9.697e^{-2}$
		200	$2.307e^{-2}$
	Positive part ratio	Overall	$1.158e^{-3}$
		50	$4.979e^{-2}$
		75	$4.724e^{-1}$
		75 7-point	$2.944e^{-2}$
		100	$4.979e^{-2}$
		150	$4.412e^{-1}$
		200	$2.231e^{-1}$

Table 6. Statistical significance results for each difficulty parameter

$ \mathcal{C} $	door	letter	logic	room	social	team
5	94	135	92	86	102	115
10	139	204	134	129	149	176
20	258	387	239	240	274	333
50	618	925	563	578	640	799
75	920	1380	841	861	951	1188
100	1218	1829	1112	1140	1256	1576
150	1844	2743	1665	1729	1882	2365
200	2470	3643	2216	2317	2496	3140

Table 7. Estimated mean token count per prompt ($\lfloor \text{chars}/4 \rfloor$) as a function of $|\mathcal{C}|$, for the six template verbalizers.

Generator	$ \mathcal{C} $	Template	LLM	Ratio
ImplicationCycle	5	269	1 277	4.75×
	10	340	2 138	6.28×
	15	413	3 084	7.47×
	20	484	4 039	8.34×
EquivalenceCore	5	240	945	3.94×
	10	325	1 999	6.15×
	15	384	2 690	7.01×
	20	469	3 766	8.03×
Backbone	10	312	1 989	6.38×
	20	470	3 757	7.99×
MonoBridge	10	324	2 073	6.40×
	20	469	3 900	8.32×
	50	896	9 307	10.39×

Table 8. Measured mean prompt-token count (LiteLLM accounting, GPT-OSS-120B tokenizer) for template vs. LLM verbalizations at matched $|\mathcal{C}|$. Only the (generator, $|\mathcal{C}|$) pairs available in both normal and augmented runs are shown.

P. Tool Calling

Models equipped with code-execution tools have access to off-the-shelf SAT solvers, which leaves open whether the failures documented in Table 1 stem from reasoning competence per se or from the absence of an external solver. To probe this, we evaluate GPT-5.4-mini on 110 items (`logic` verbalizer only) in two configurations: (i) *without tools*; (ii) *with an `execute_python` tool* exposing the `pysat` solver (reasoning disabled, due to an API constraint). The model is free to choose whether to invoke the tool.

Findings. (i) Tool calling provides large gains where the bottleneck is search (EquivalenceCore: +60 pp; MonoBridge: +17 pp), since the model offloads the assignment search to `pysat`. (ii) Tool calling does *not* help on ImplicationCycle (−2.9 pp): the model invokes the tool 94% of the time, but the bottleneck lies in correctly translating the natural-language clauses into a CNF the solver can ingest. (iii) Tool calling complements rather than replaces the reasoning evaluation: it isolates the *NL*→*CNF* parsing step as a residual failure mode, distinct from the search and propagation failures targeted by the main benchmark.

Model	TP	FP	TN	FN	Acc (%)
Qwen3-Next-80B	881	44	374	4	96.3
Phi-4-reasoning-plus	695	85	135	3	90.4
GPT-OSS-120B	877	111	309	21	90.0
OLMo-3-32B	824	91	327	39	89.9
Phi-4-reasoning	804	140	165	14	86.3
QwQ-32B	818	143	277	35	86.0
Llama-3.3-70B-Instruct	886	371	48	13	70.9

Table 9. Per-model confusion matrix for the SAT/UNSAT decision over the full evaluation set. TP/FP/TN/FN are absolute counts; Acc is the overall decision accuracy.

Generator	C	o3-mini		o4-mini		GPT-5.4-mini	
		Sat.	Wit.	Sat.	Wit.	Sat.	Wit.
ImplicationCycle	5	100.0	—	98.3	—	100.0	—
	10	100.0	—	91.7	—	91.7	—
	15	96.7	—	85.0	—	96.7	—
	20	88.3	—	81.7	—	91.7	—
	50	75.0	—	63.3	—	65.0	—
	75	50.0	—	65.0	—	48.3	—
	100	50.0	—	60.0	—	43.3	—
EquivalenceCore	10	100.0	83.3	100.0	80.0	100.0	65.0
	15	100.0	36.7	100.0	38.3	100.0	45.0
	20	100.0	40.0	100.0	48.3	98.3	36.7
	50	100.0	3.3	100.0	15.0	100.0	20.0
Backbone	20	100.0	100.0	100.0	100.0	100.0	100.0
	50	100.0	85.0	100.0	93.3	100.0	100.0
	75	100.0	86.7	98.3	80.0	100.0	95.0
	100	100.0	78.3	100.0	86.7	100.0	95.0
MonoBridge	10	100.0	100.0	100.0	98.3	100.0	98.3
	20	100.0	98.3	100.0	93.3	100.0	93.3
	50	100.0	81.7	100.0	75.0	98.3	86.7
	75	100.0	70.0	96.7	56.7	100.0	81.7
	100	100.0	51.7	83.3	46.7	98.3	71.7
	150	100.0	40.0	53.3	8.3	90.0	38.3
	200	100.0	28.3	48.3	11.7	70.0	13.3

Table 10. Decision accuracy (Sat., %) and witness validity (Wit., %, SAT instances only) for three proprietary reasoning models. Each cell averages 10 formulas $\times$ 6 template verbalizations, matching the protocol of Table 1. The combined micro-average (witness validity for SAT generators, decision accuracy for ImplicationCycle) is 70.2% (o3-mini), 67.1% (o4-mini) and 71.7% (GPT-5.4-mini).

Generator	C	Low		Medium		High	
		Sat.	Wit.	Sat.	Wit.	Sat.	Wit.
ImplicationCycle	5	98.3	—	91.7	—	75.0	—
	10	91.7	—	95.0	—	80.0	—
	15	76.7	—	93.3	—	88.3	—
	20	51.7	—	93.3	—	78.3	—
	50	40.0	—	56.7	—	30.0	—
	75	21.7	—	45.0	—	18.3	—
	100	28.3	—	40.0	—	8.3	—
EquivalenceCore	10	100.0	31.7	100.0	83.3	100.0	93.3
	15	100.0	10.0	100.0	46.7	100.0	55.0
	20	98.3	6.7	100.0	50.0	98.3	63.3
	50	100.0	0.0	100.0	8.3	98.3	31.7
Backbone	20	100.0	100.0	100.0	96.7	100.0	100.0
	50	100.0	91.7	100.0	98.3	100.0	100.0
	75	100.0	88.3	98.3	88.3	100.0	98.3
	100	100.0	78.3	100.0	83.3	100.0	95.0
MonoBridge	10	100.0	95.0	100.0	100.0	100.0	100.0
	20	100.0	80.0	100.0	100.0	100.0	98.3
	50	86.7	51.7	100.0	83.3	100.0	93.3
	75	70.0	28.3	100.0	68.3	100.0	83.3
	100	63.3	20.0	100.0	53.3	93.3	58.3
	150	53.3	6.7	90.0	31.7	81.7	35.0
	200	58.3	5.0	73.3	23.3	58.3	11.7

Table 11. GPT-OSS-120B decision accuracy (Sat., %) and witness validity (Wit., %, SAT instances only) at three reasoning-effort levels.

Generator	Without tools	With tools (% code used)
Backbone	85.0	90.0 (90%)
EquivalenceCore	30.0	90.0 (90%)
ImplicationCycle	74.3	71.4 (94%)
MonoBridge	82.9	100.0 (100%)
Overall	70.9	87.3 (95%)

Table 12. GPT-5.4-mini accuracy (%) without vs. with code-execution tool ($n = 110$ items, `logic` verbalizer). The parenthesised number is the proportion of items where the model chose to invoke the tool.